School of Computer Science Engineering and Information Systems
Winter Semester 2025-26
Agile Development Process and Devops Lab
Digital Assignment - II

NAME : ARUNIMA SANJEEV
REG NO : 23MIS0048

1. Case Study 1: New Project Version Control Setup

```
library_management_system
Welcome    library.py
library.py
1   print("Library Management System")

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                zsh
arunimasanjeev@Arunimas-MacBook-Air library_management_system % git init
Initialized empty Git repository in /Users/arunimasanjeev/Desktop/library_management_system/.git/
arunimasanjeev@Arunimas-MacBook-Air library_management_system % git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        library.py

nothing added to commit but untracked files present (use "git add" to track)
arunimasanjeev@Arunimas-MacBook-Air library_management_system % git add library.py
arunimasanjeev@Arunimas-MacBook-Air library_management_system % git commit -m"Initial commit: Library Management System"
[main (root-commit) dd6c2db] Initial commit: Library Management System
 1 file changed, 1 insertion(+)
 create mode 100644 library.py
arunimasanjeev@Arunimas-MacBook-Air library_management_system %
```

```
arunimasanjeev@Arunimas-MacBook-Air library_management_system % git diff
diff --git a/library.py b/library.py
index 625ccf4..14c0322 100644
--- a/library.py
+++ b/library.py
@@ -1 +1,2 @@
-print("Library Management System")
\ No newline at end of file
+print("Library Management System")
+print("hello")
\ No newline at end of file
arunimasanjeev@Arunimas-MacBook-Air library_management_system % git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   library.py

no changes added to commit (use "git add" and/or "git commit -a")
arunimasanjeev@Arunimas-MacBook-Air library_management_system % git add library.py

arunimasanjeev@Arunimas-MacBook-Air library_management_system % git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   library.py

arunimasanjeev@Arunimas-MacBook-Air library_management_system % git commit -m "Added greeting message to library modul
e"
[main af9ae1b] Added greeting message to library module
 1 file changed, 2 insertions(+), 1 deletion(-)
arunimasanjeev@Arunimas-MacBook-Air library_management_system % git log --oneline
af9ae1b (HEAD -> main) Added greeting message to library module
dd6c2db Initial commit: Library Management System
```

2. Case Study 2: Tracking Daily Code Changes

Initialize project and Git

```
arunimasanjeev@Arunimas-MacBook-Air ~ % mkdir StudentRegistrationModule
cd StudentRegistrationModule
```

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git init
```

Create initial file

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % cat > student.py << EOF
def register_student(name, age):
    print(f"Student {name} registered, Age: {age}")
EOF
```

First commit

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git add .
git commit -m "Initial project setup"

[main (root-commit) c01f979] Initial project setup
 4 files changed, 12 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 database.py
 create mode 100644 student.py
```

Modify existing file

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % cat >> student.py << EOF

def delete_student(name):
    print("Student deleted:", name)
EOF
```

Check differences:

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git diff
diff --git a/student.py b/student.py
index 211333a..8024674 100644
--- a/student.py
+++ b/student.py
@@ -1,2 +1,5 @@
 def register_student(name, age):
     print(f"Student {name} registered, Age: {age}")
+
+def delete_student(name):
+    print("Student deleted:", name)
```

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git status
git diff
git add .
git commit -m "Added update_student function"
git log --oneline

On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   student.py
```

Stage and commit:

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git add student.py

arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git commit -m "Added delete_student function"

[main 51f66d3] Added delete_student function
 1 file changed, 3 insertions(+)
```

View commit history

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git log --oneline

51f66d3 (HEAD -> main) Added delete_student function
732d605 Added test.py for testing
c01f979 Initial project setup
```

3. Case Study 3: Feature Development Using Branches

Check current branch

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git branch
* main
```

Create a new branch and switch to that branch

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git branch login-feature
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git switch login-feature
Switched to branch 'login-feature'
```

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git branch
* login-feature
  main
```

Add login-related code

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % cat > login.py << EOF
def login(username, password):
    if username == "admin" and password == "1234":
        print("Login successful")
    else:
        print("Invalid credentials")
EOF
```

Stage and commit changes

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git status
On branch login-feature
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        login.py
        output.txt

nothing added to commit but untracked files present (use "git add" to track)
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git add login.py
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git commit -m "Added login feature"
[login-feature a0ab50f] Added login feature
 1 file changed, 5 insertions(+)
 create mode 100644 login.py
```

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git log --oneline
a0ab50f (HEAD -> login-feature) Added login feature
4421e14 (main) Added update_student function
51f66d3 Added delete_student function
732d605 Added test.py for testing
c01f979 Initial project setup
```

Switch back to main branch

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git switch main
Switched to branch 'main'
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % ls
README.md        database.py      output.txt       student.py       test.py
```

Switch again to feature branch

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git switch login-feature
Switched to branch 'login-feature'
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % ls
README.md        database.py      login.py         output.txt       student.py       test.py
```

4. **Case Study 4: Merging Feature into Main Code**

Switch to main branch

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git switch main

Switched to branch 'main'
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git branch
  login-feature
* main
```

Merge feature branch into main

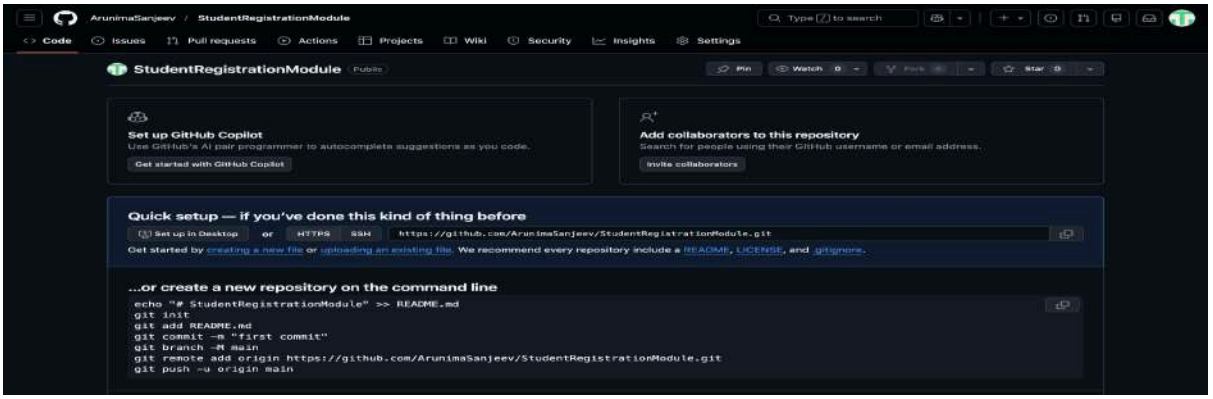
```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git merge login-feature
Updating 4421e14..a0ab50f
Fast-forward
 login.py | 5 +++++
 1 file changed, 5 insertions(+)
 create mode 100644 login.py
```

Verify merged files

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % ls
README.md        database.py      login.py         output.txt       student.py       test.py
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git log --oneline
a0ab50f (HEAD -> main, login-feature) Added login feature
4421e14 Added update_student function
51f66d3 Added delete_student function
732d605 Added test.py for testing
c01f979 Initial project setup
```

5. **Case Study 5: Working with Remote Repository**

Create remote repository



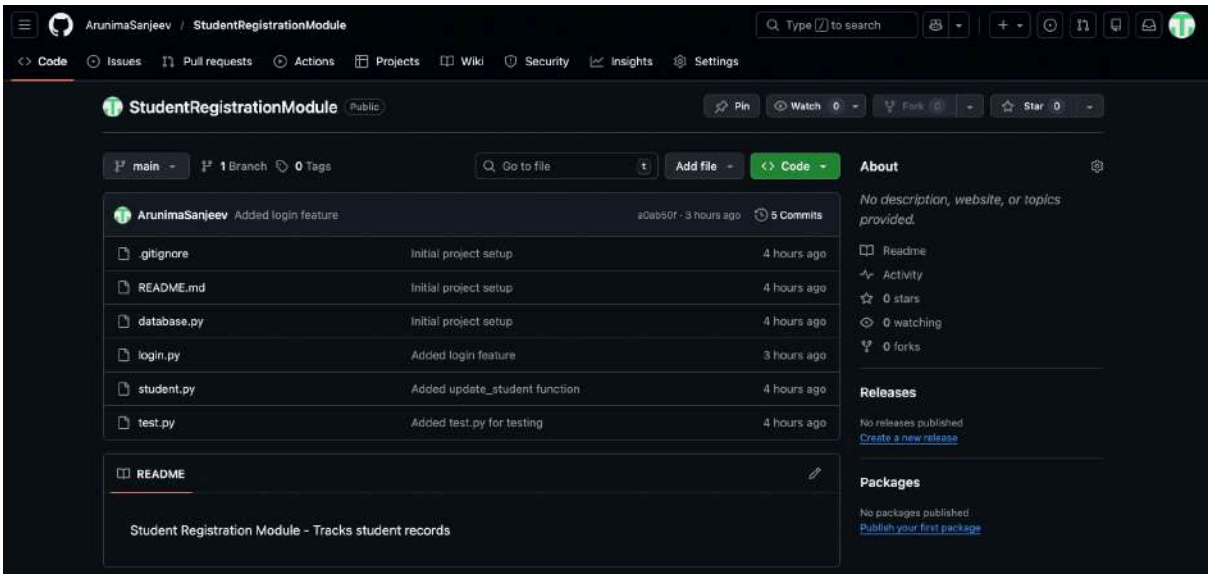
Add remote origin and verify

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git remote add origin https://github.com/ArunimaSanjeev/StudentRegistrationModule
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git remote -v
origin  https://github.com/ArunimaSanjeev/StudentRegistrationModule (fetch)
origin  https://github.com/ArunimaSanjeev/StudentRegistrationModule (push)
```

Push code to GitHub

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git push -u origin main
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 8 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (18/18), 1.63 KiB | 1.63 MiB/s, done.
Total 18 (delta 6), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/ArunimaSanjeev/StudentRegistrationModule
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Verify on GitHub



6. Case Study 6: Undoing Mistakes

Modify file incorrectly

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % cat >> student.py << EOF

print("WRONG CODE ADDED")
EOF
```

Check repository status

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git status

On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   student.py
```

View file differences

```
[arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git diff
diff --git a/student.py b/student.py
index 571c009..9d633be 100644
--- a/student.py
+++ b/student.py
@@ -6,3 +6,5 @@ def delete_student(name):

 def update_student(name, age):
     print(f"Updated {name}, age {age}")
+
+print("WRONG CODE ADDED")
```

Discard uncommitted changes

```
[arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git restore student.py
[arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git status
On branch main
Your branch is up to date with 'origin/main'.
```

7. **Case Study 7: Handling Merge Conflicts**

Create two branches

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git switch -c feature-A

Switched to a new branch 'feature-A'
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % cat >> student.py << EOF

print("Feature A change")
EOF

arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git add student.py
git commit -m "Feature A update"

[feature-A 8742018] Feature A update
 1 file changed, 2 insertions(+)
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git switch main
git switch -c feature-B

Switched to branch 'main'
Your branch is up to date with 'origin/main'.
Switched to a new branch 'feature-B'
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % cat >> student.py << EOF

print("Feature B change")
EOF

arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git add student.py
git commit -m "Feature B update"

[feature-B bbbb6cc] Feature B update
 1 file changed, 2 insertions(+)
```

Merge branches

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git switch main

Switched to branch 'main'
Your branch is up to date with 'origin/main'.
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git merge feature-A

Updating a0ab50f..8742018
Fast-forward
 student.py | 2 ++
 1 file changed, 2 insertions(+)
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git merge feature-B

Auto-merging student.py
CONFLICT (content): Merge conflict in student.py
Automatic merge failed; fix conflicts and then commit the result.
```

Open conflicted file

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % cat student.py
def register_student(name, age):
    print(f"Student {name} registered, Age: {age}")

def delete_student(name):
    print("Student deleted:", name)

def update_student(name, age):
    print(f"Updated {name}, age {age}")

<<<<<<< HEAD
print("Feature A change")
=======
print("Feature B change")
>>>>>>> feature-B
```

Resolve conflict manually and stage

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % print("Feature A change")
print("Feature B change")

zsh: missing delimiter for 'g' glob qualifier
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git add student.py
```

Commit resolved changes

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git commit -m "Resolved merge conflict between feature-A and feature-B"

[main b56334b] Resolved merge conflict between feature-A and feature-B
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git log --oneline

b56334b (HEAD -> main) Resolved merge conflict between feature-A and feature-B
bbbb6cc (feature-B) Feature B update
8742018 (feature-A) Feature A update
a0ab50f (origin/main) Added login feature
4421e14 Added update_student function
51f66d3 Added delete_student function
732d605 Added test.py for testing
c01f979 Initial project setup
```

8. Case Study 8: Ignoring Unwanted Files

Create unwanted files

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % touch app.log
touch debug.log
touch temp.txt
mkdir __pycache__
```

Check Git status

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git status

On branch main
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        app.log
        debug.log
        output.txt
        temp.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Create .gitignore file

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % touch .gitignore
```

Add ignore patterns

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % cat > .gitignore << EOF
# Log files
*.log

# Temporary files
*.txt

# Python cache
__pycache__/
*.pyc

# Environment files
.env

# IDE files
.vscode/
.idea/
EOF
```

Verify ignored files

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git status

On branch main
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

no changes added to commit (use "git add" and/or "git commit -a")
```

Commit .gitignore

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git add .gitignore
git commit -m "Added .gitignore to ignore logs and temp files"

[main 5bb5475] Added .gitignore to ignore logs and temp files
 1 file changed, 11 insertions(+)
```

9. Case Study 9: Collaborating with Team

Check current repository status

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git status

On branch main
Your branch is ahead of 'origin/main' by 4 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

Fetch latest changes from remote repository

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git fetch origin
```

View new commits

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git log origin/main --oneline

a0ab50f (origin/main) Added login feature
4421e14 Added update_student function
51f66d3 Added delete_student function
732d605 Added test.py for testing
c01f979 Initial project setup
```

Pull updates into local branch

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git pull origin main

From https://github.com/ArunimaSanjeev/StudentRegistrationModule
 * branch            main       -> FETCH_HEAD
Already up to date.
```

Verify updated files

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % ls
README.md    __pycache__    app.log    database.py    debug.log    login.py    output.txt    student.py    temp.txt    test.py
```

Resolve conflicts (if any)

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % cat student.py

def register_student(name, age):
    print(f"Student {name} registered, Age: {age}")

def delete_student(name):
    print("Student deleted:", name)

def update_student(name, age):
    print(f"Updated {name}, age {age}")

<<<<<<< HEAD
print("Feature A change")
=======
print("Feature B change")
>>>>>>> feature-B
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git add student.py
git commit -m "Resolved merge conflict after pull"

On branch main
Your branch is ahead of 'origin/main' by 4 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

10. Case Study 10: Final Project Release

Review commit history

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git log --oneline

5bb5475 (HEAD -> main) Added .gitignore to ignore logs and temp files
b56334b Resolved merge conflict between feature-A and feature-B
bbbb6cc (feature-B) Feature B update
8742018 (feature-A) Feature A update
a0ab50f (origin/main) Added login feature
4421e14 Added update_student function
51f66d3 Added delete_student function
732d605 Added test.py for testing
c01f979 Initial project setup
```

Add final changes (if any)

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git add .
git commit -m "Final project version ready for release"

On branch main
Your branch is ahead of 'origin/main' by 4 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

Push final code to GitHub

```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git push origin main

Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 1.19 KiB | 1.19 MiB/s, done.
Total 12 (delta 8), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (8/8), completed with 2 local objects.
To https://github.com/ArunimaSanjeev/StudentRegistrationModule
   a0ab50f..5bb5475  main -> main
```

Create release version(tag)

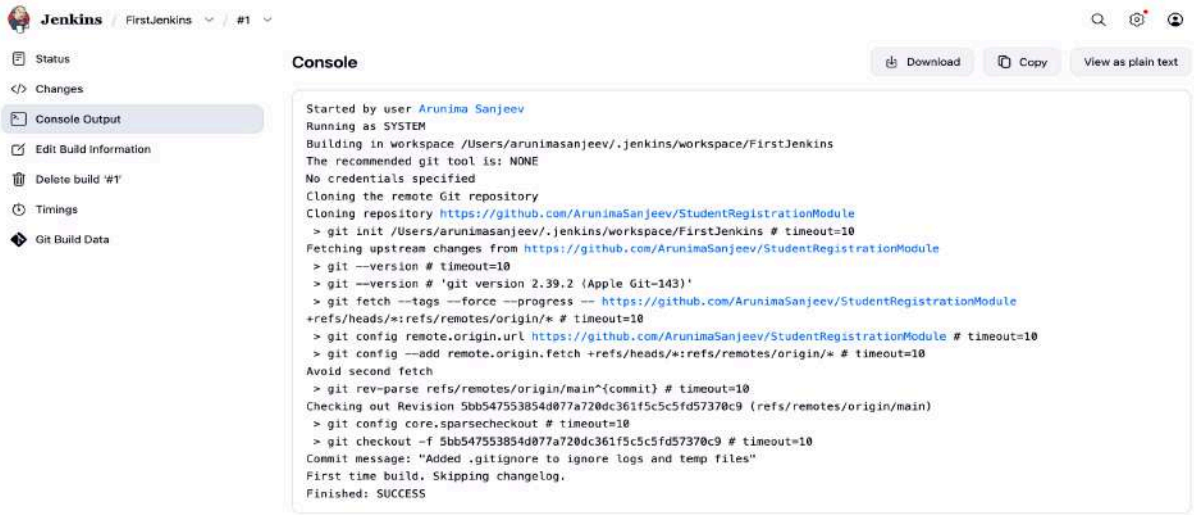
```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git tag -a v1.0 -m "Final Release Version 1.0"

arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git push origin v1.0

Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 179 bytes | 179.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/ArunimaSanjeev/StudentRegistrationModule
 * [new tag]         v1.0 -> v1.0
```

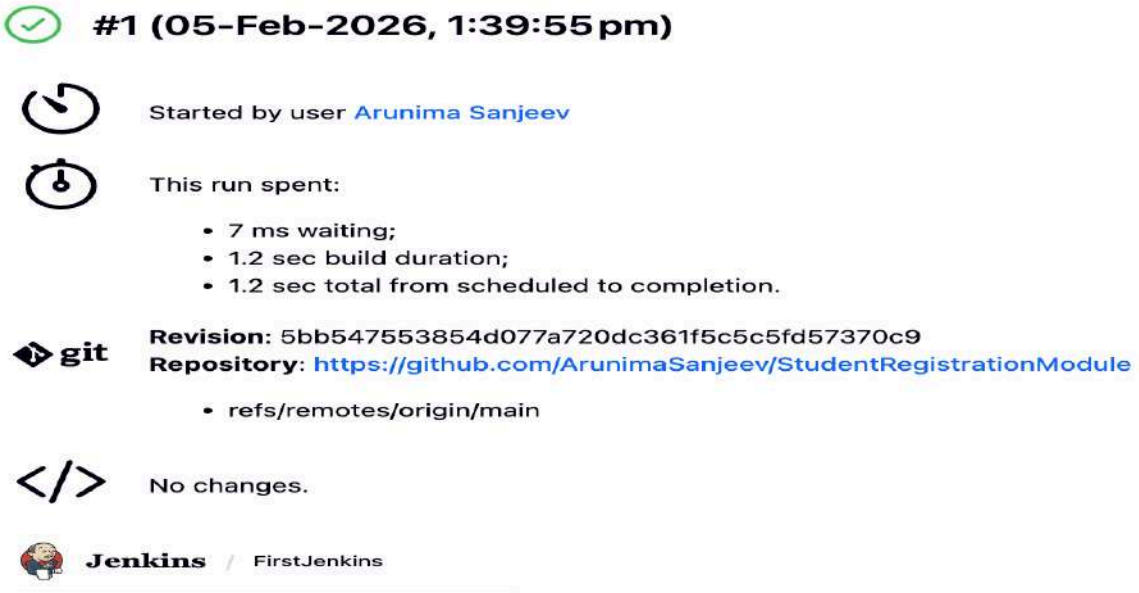
Verify tag

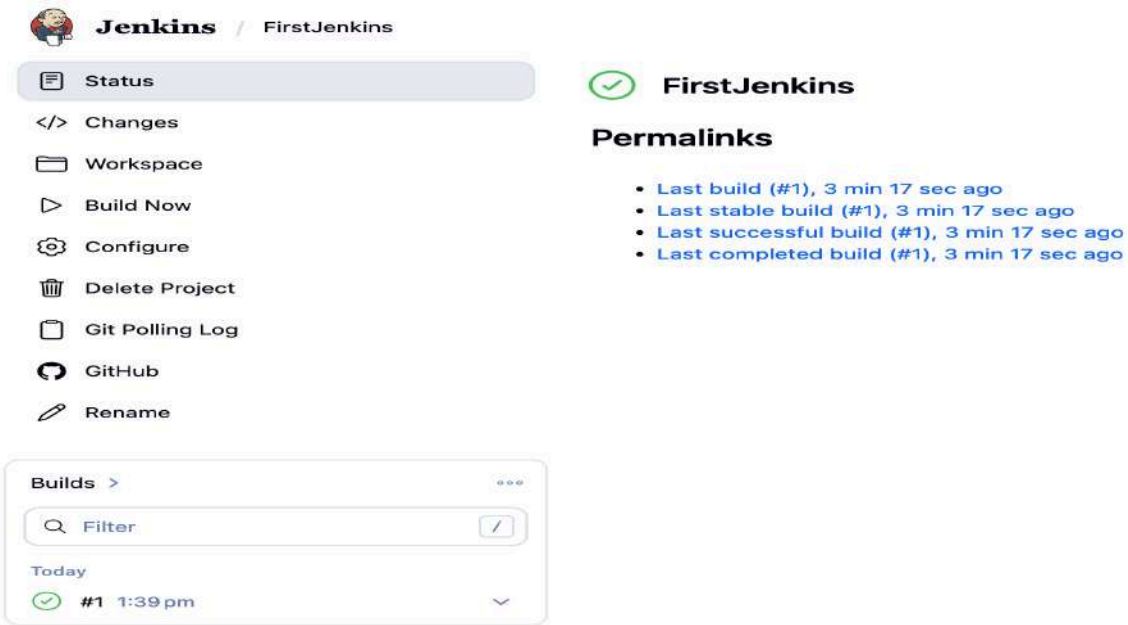
```
arunimasanjeev@Arunimas-MacBook-Air StudentRegistrationModule % git tag

v1.0
```

JENKINS

1. Create a Job







New Item

Enter an item name

JenkinsFirstJob1

Select an item type

Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.

Organization Folder

OK

- [ ] Discard old builds ?

- [x] GitHub project

Project url ?

https://github.com/ArunimaSanjeev/StudentRegistrationModule

Advanced

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

- ( ) None
- (•) Git ?

Repositories ?

Repository URL ?

https://github.com/ArunimaSanjeev/StudentRegistrationModule

Credentials ?

- none - + Add

Advanced

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- [ ] Trigger builds remotely (e.g., from scripts) ?
- [ ] Build after other projects are built ?
- [ ] Build periodically ?
- [ ] GitHub hook trigger for GITScm polling ?
- [x] Poll SCM ?

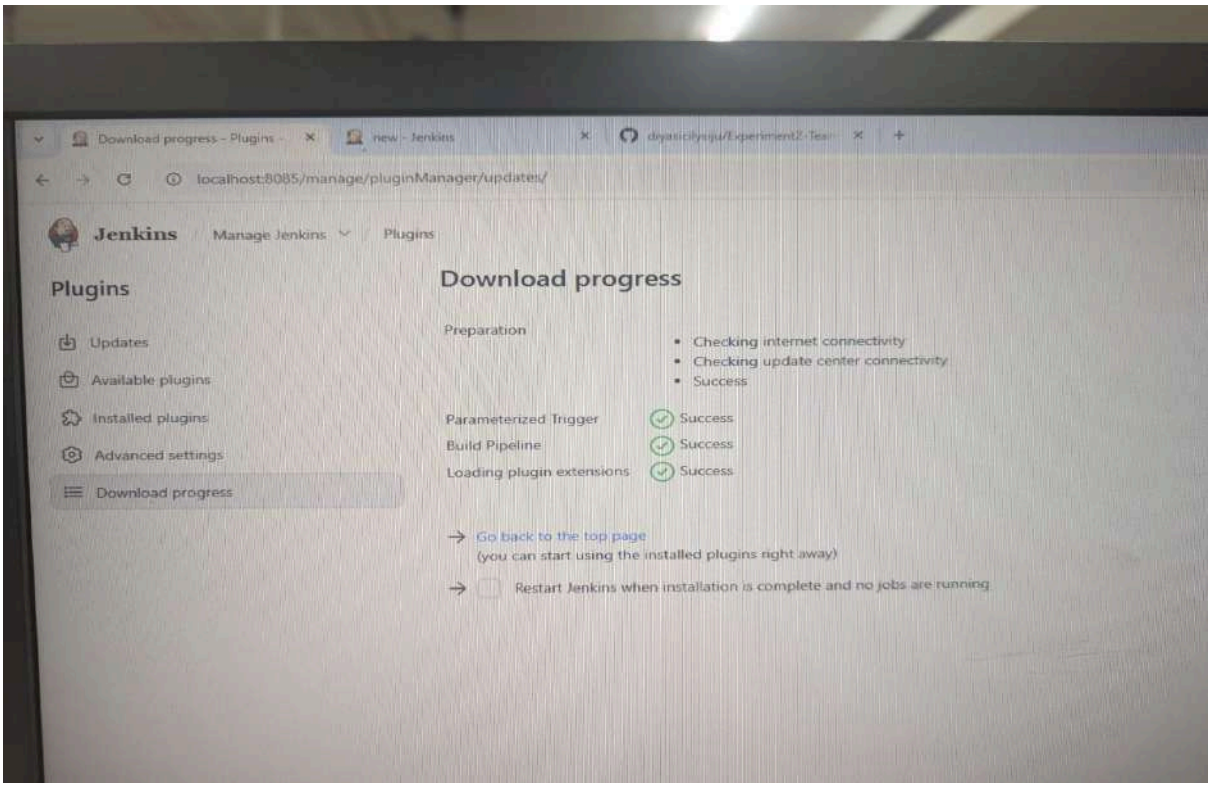
Schedule ?

```
H/1 * * * *
```

Would last have run at Thursday, 5 February, 2026, 1:20:00 pm India Standard Time; would next run at Thursday, 5 February, 2026, 2:20:00 pm India Standard Time.

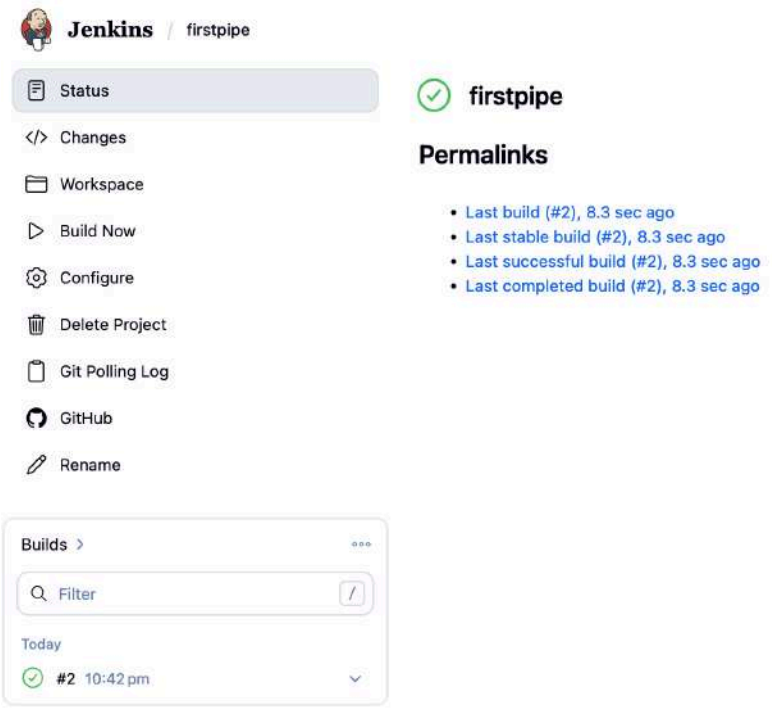
- [ ] Ignore post-commit hooks ?

Install Plugin



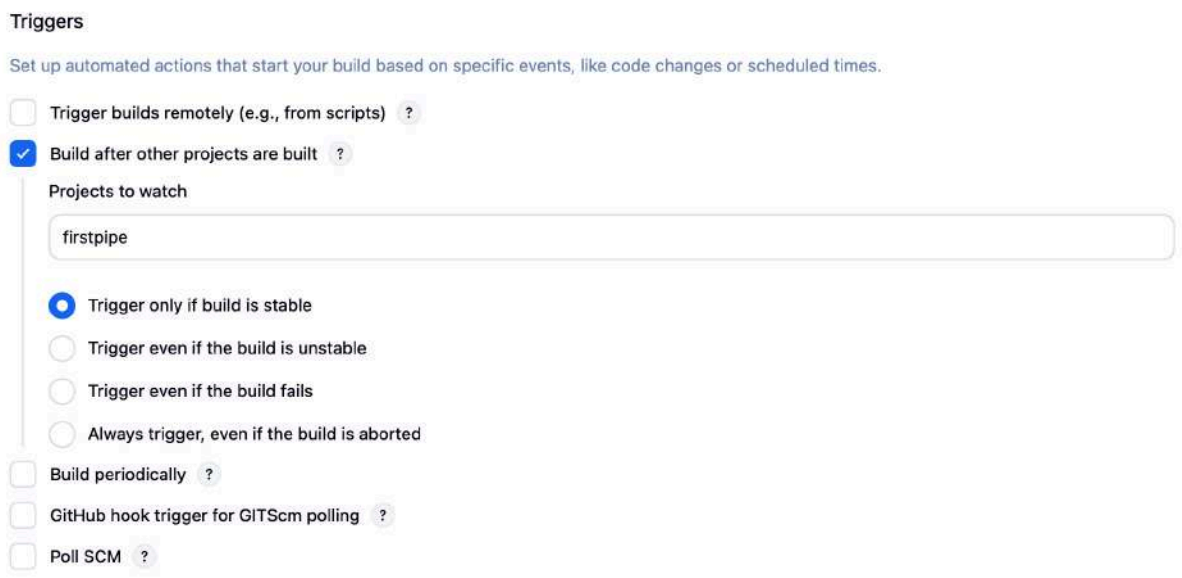
CREATE PIPELINE

Create a job named "firstpipe"
Fill the necessary blanks, click apply and save
Click build now

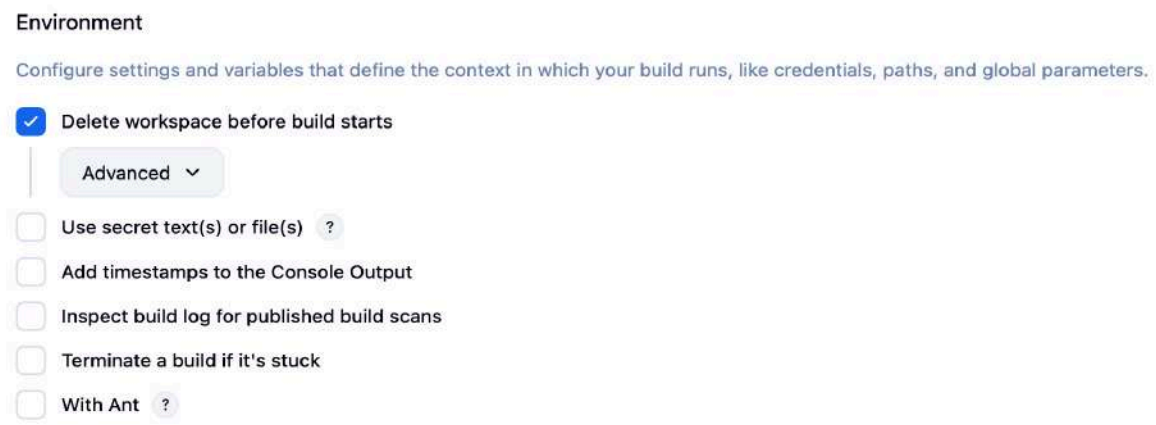


To create the second job named “secondpipe”
Repeat the same steps as above with the following changes:

i. Select “Build after other projects are built” under Triggers section and type the first job’s name created. Here, type “firstpipe”.



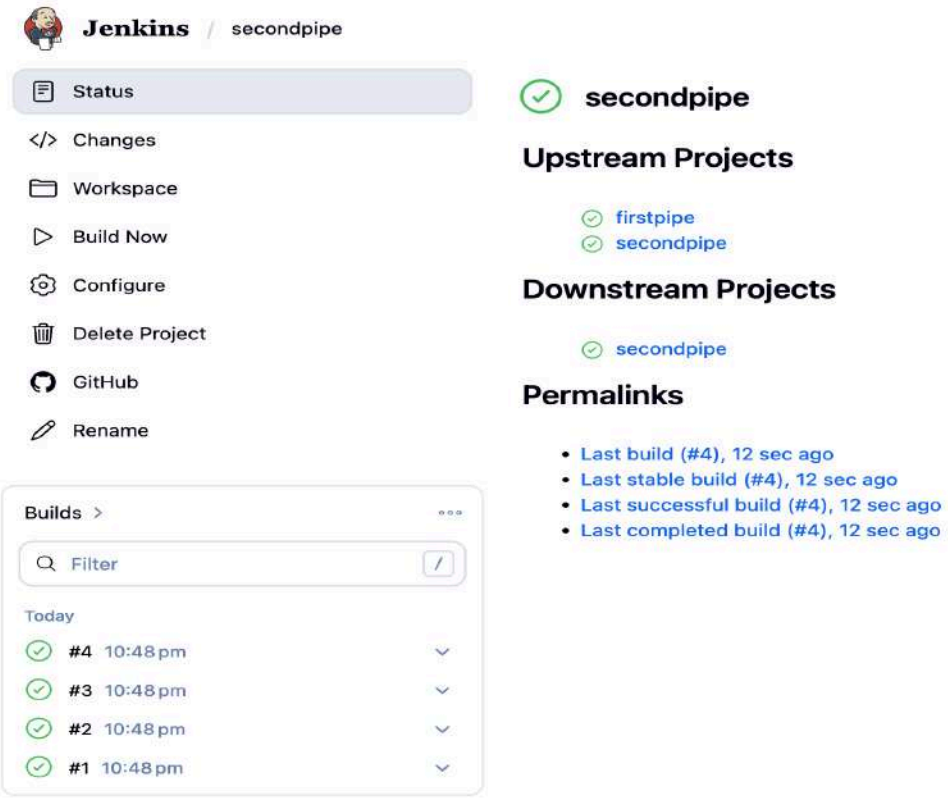
ii. . Select the “Delete workspace before build starts” option under the Environment section.



iii. Select Execute Shell command (for mac) or Execute Windows batch command option under Build Steps. Enter the command for execution of the file in the repository.

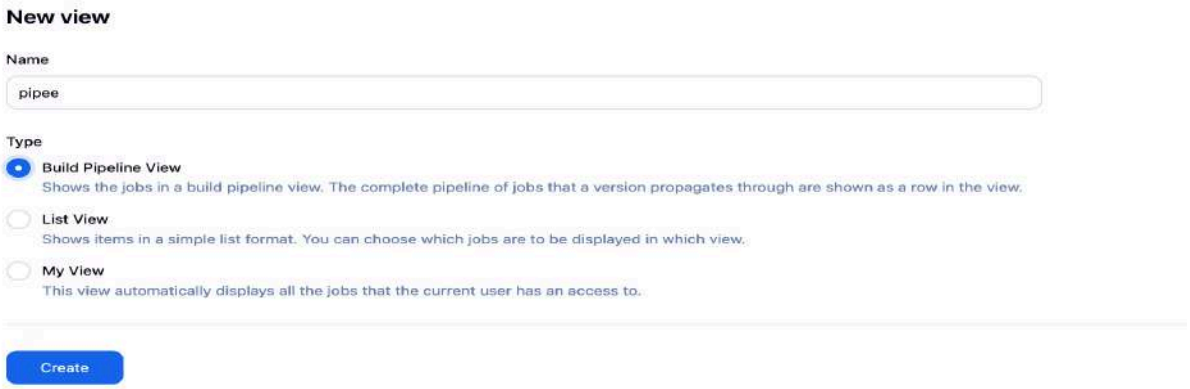
iv. Select "secondpipe" as the project to build under Post-build actions section

Click Apply and Save
Click Build now

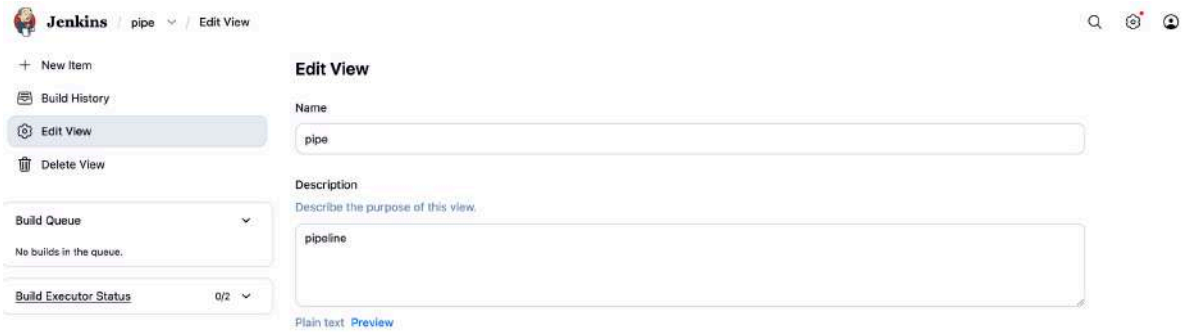


BUILD PIPELINE:

i. Enter the name of the pipeline. Select “Build pipeline view” type.

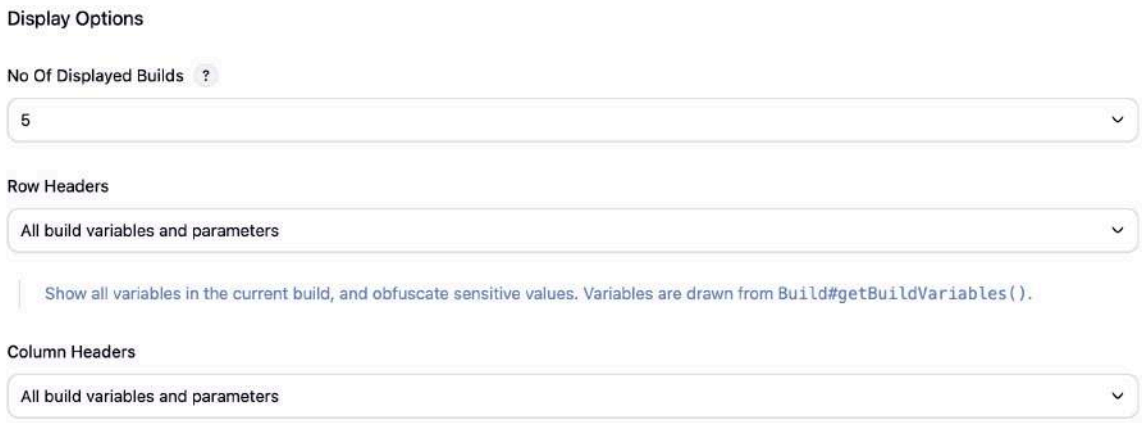


ii. Type name and description for the pipeline under the description section.

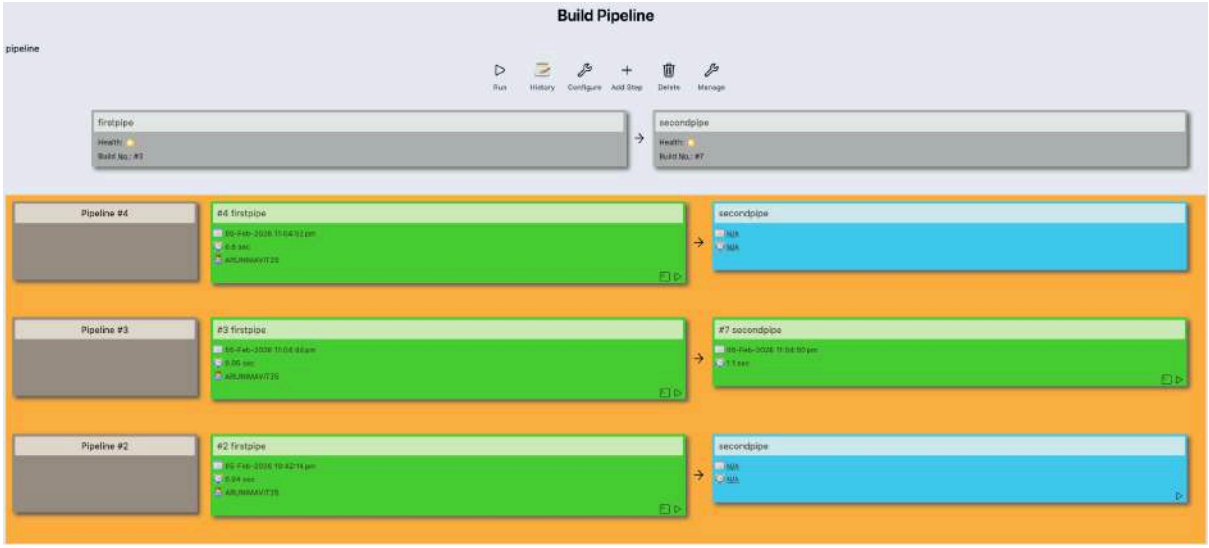


iii. Select the Layout and Initial Job

iv. Select number of builds to be displayed, row and column headers

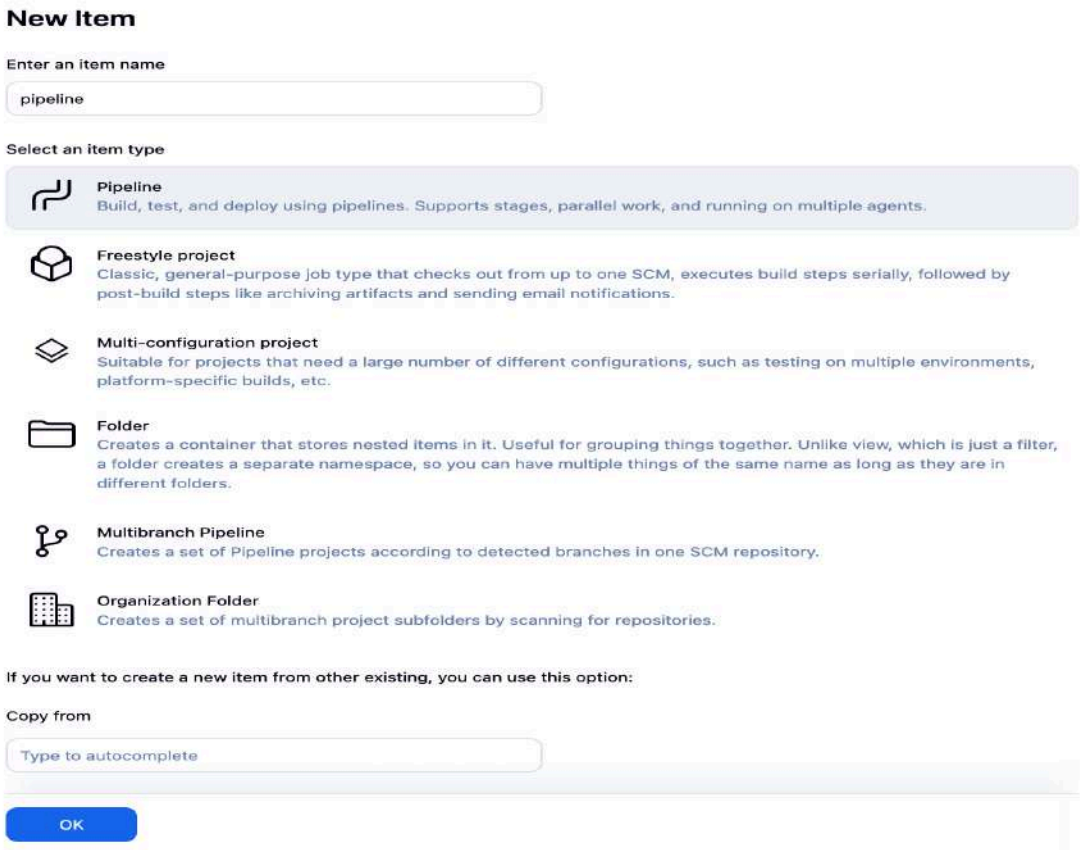


v. Click Apply and Save
Click Run button



PIPELINE JOB CREATION

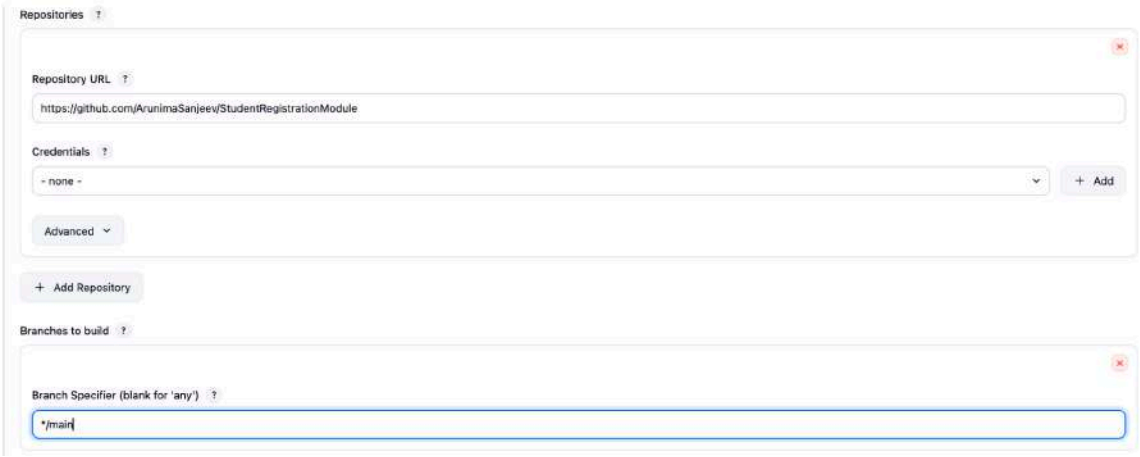
i. Enter pipeline name and select “pipeline” as item type. Click OK.



ii. Under the Pipeline section select Definition Script type and type the script

iii. Click on the pipeline syntax option

iv. Select checkout:Check out from version control option. Enter the Github repository URL. Select generate pipeline script option. Copy and paste it into the script.



v. Select Pipeline syntax option again. Select “Windows Batch Script” for windows (or “Shell Script” for mac) from the drop down menu.

vi. Enter dir (or “ls” for mac) in the Batch Script column

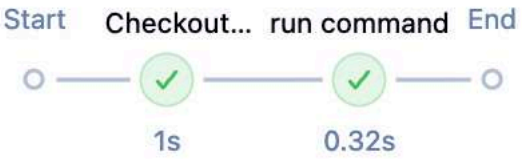
vii. Copy and paste the pipeline script into the script.
viii. Click on Apply and Save.
Click on build now option.

Stages

5 February 2026

#1

23:18 - 2s



#1 (05-Feb-2026, 11:18:45 pm)

Started by user Arunima Sanjeev

This run spent:

- 9 ms waiting;
- 2.3 sec build duration;
- 2.3 sec total from scheduled to completion.

Revision: 5bb547553854d077a720dc361f5c5c5fd57370c9
Repository: https://github.com/ArunimaSanjeev/StudentRegistrationModule

- refs/remotes/origin/main

No changes.

PIPELINE TASKS

Pipeline Task 1:

Pipeline for Git Repository Checkout and Workspace File Listing

Definition

Pipeline script

Script

```
pipeline {
  agent any
  stages {
  stage('Checkout Code') {
  steps {
  checkout scmGit(branches: [[name: '*/main']], extensions: [], userRemoteConfigs: [[url: 'https://github.com/ArunimaSanjeev/Student
  }
  }
  stage('Show Files') {
  steps {
  sh 'ls'
  }
  }
  }
}
```

Use Groovy Sandbox

Pipeline Syntax

Sample Step

checkout: Check out from version control

checkout

SCM

Git

Repositories

Repository URL

https://github.com/ArunimaSanjeev/StudentRegistrationModule

Credentials

- none -

+ Add

Advanced

+ Add Repository

Branches to build

Branch Specifier (blank for 'any')

*/main

+ Add Branch

Repository browser

(Auto)

Additional Behaviours

+ Add

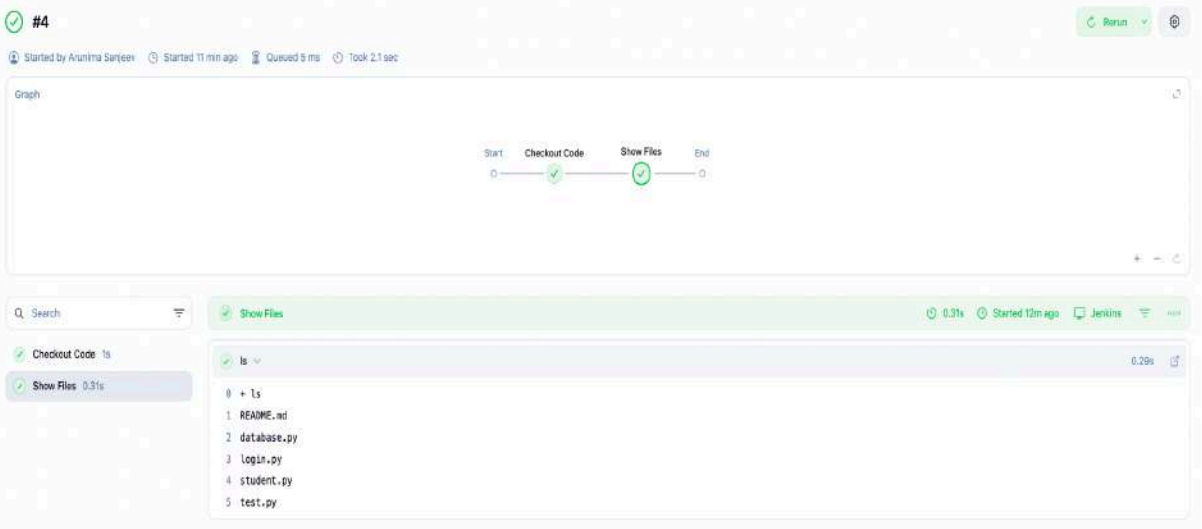
Include in polling?

Include in changelog?

Generate Pipeline Script

```
checkout scmGit(branches: [[name: '*/main']], extensions: [], userRemoteConfigs: [[url: 'https://github.com/ArunimaSanjeev/StudentRegistrationModule']])
```

```
Running on Jenkins in /Users/arunimasanjeev/.jenkins/workspace/pipeone
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Code)
[Pipeline] checkout
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/ArunimaSanjeev/StudentRegistrationModule
 > git init /Users/arunimasanjeev/.jenkins/workspace/pipeone # timeout=10
Fetching upstream changes from https://github.com/ArunimaSanjeev/StudentRegistrationModule
 > git --version # timeout=10
 > git --version # 'git version 2.39.2 (Apple Git-143)'
 > git fetch --tags --force --progress -- https://github.com/ArunimaSanjeev/StudentRegistrationModule +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/ArunimaSanjeev/StudentRegistrationModule # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5bb547553854d077a720dc361f5c5c5fd57370c9 (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 5bb547553854d077a720dc361f5c5c5fd57370c9 # timeout=10
Commit message: "Added .gitignore to ignore logs and temp files"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Show Files)
[Pipeline] sh
+ ls
README.md
database.py
login.py
student.py
test.py
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Pipeline Task 2:

Pipeline for Git Repository Checkout and Workspace Directory Display

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1  pipeline {
2    agent any
3    stages {
4    stage('Checkout Code') {
5    steps {
6    checkout scmGit(branches: [[name: '*/main']], extensions: [], userRemoteConfigs: [[url: 'https://github.com/ArunimaSanjeev/StudentRegistrati
7    }
8    }
9    stage('Print Directory') {
10   steps {
11   sh 'pwd'
12   }
13   }
14   }
15 }
```

Use Groovy Sandbox ?

Pipeline Syntax

Jenkins / pipetwo

- Status
- Changes
- Build Now
- Configure
- Delete Pipeline
- Stages
- Rename
- Pipeline Syntax

Builds >

Filter /

Today

#2 12:07 pm

pipetwo

Permalinks

- Last build (#2), 10 sec ago
- Last stable build (#2), 10 sec ago
- Last successful build (#2), 10 sec ago
- Last completed build (#2), 10 sec ago

Console

Download Copy

```
Started by user Arunima Sanjeev
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /Users/arunimasanjeev/.jenkins/workspace/pipetwo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Code)
[Pipeline] checkout
The recommended git tool is: NONE
No credentials specified
 > git rev-parse --resolve-git-dir /Users/arunimasanjeev/.jenkins/workspace/pipetwo/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/ArunimaSanjeev/StudentRegistrationModule # timeout=10
Fetching upstream changes from https://github.com/ArunimaSanjeev/StudentRegistrationModule
 > git --version # timeout=10
 > git --version # 'git version 2.39.2 (Apple Git-143)'
 > git fetch --tags --force --progress -- https://github.com/ArunimaSanjeev/StudentRegistrationModule +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5bb547553854d077a720dc361f5c5c5fd57370c9 (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 5bb547553854d077a720dc361f5c5c5fd57370c9 # timeout=10
Commit message: "Added .gitignore to ignore logs and temp files"
 > git rev-list --no-walk 5bb547553854d077a720dc361f5c5c5fd57370c9 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Print Directory)
[Pipeline] sh
+ pwd
/Users/arunimasanjeev/.jenkins/workspace/pipetwo
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

#2

Rerun

Started by Arunima Sanjeev | Started 49 sec ago | Queued 2 ms | Took 1.5 sec

Graph

Start → Checkout Code → Print Directory → End

Search

Checkout Code 0.85s

Print Directory 0.32s

Print Directory — 0.32s | Started 48s ago | Jenkins

pwd — 0.29s

```
+ pwd
/Users/arunimasanjeev/.jenkins/workspace/pipetwo
```

<u>Pipeline task 3</u>

Pipeline to Clone GitHub Repository and Display Execution Confirmation Message

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
pipeline {
  agent any
  stages {
  stage('Checkout Code') {
  steps {
  checkout scmGit(branches: [[name: '*/main']], extensions: [], userRemoteConfigs: [[url: 'https://github.com/ArunimaSanjeev/StudentRegistrati
  }
  }
  stage('Print Message') {
  steps {
  echo 'Hello! Jenkins Pipeline executed successfully'
  }
  }
  }
}
```

☑ Use Groovy Sandbox ?

Jenkins / pipethree / #1

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#1'
- Timings
- Git Build Data
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces

#1 (07-Feb-2026, 12:14:02pm)

Started by user Arunima Sanjeev

This run spent:

- 7 ms waiting;
- 1.6 sec build duration;
- 1.6 sec total from scheduled to completion.

Revision: 5bb547553854d077a720dc361f5c5c5fd57370c9
Repository: https://github.com/ArunimaSanjeev/StudentRegistrationModule

- refs/remotes/origin/main

No changes.

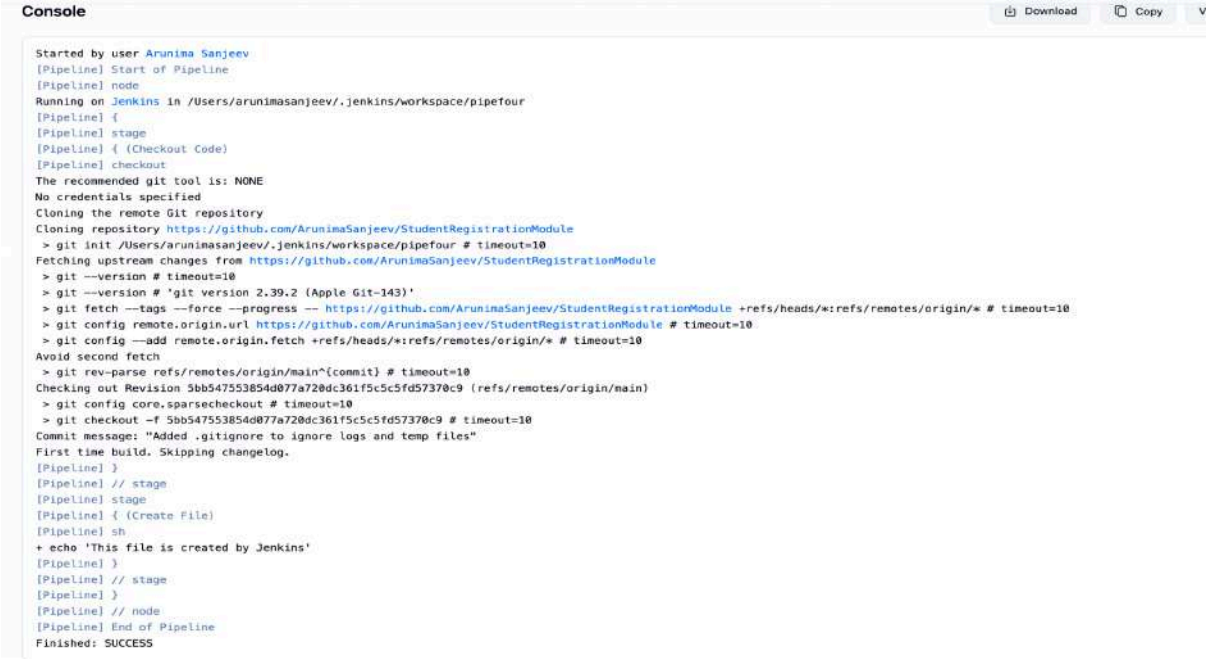
Console

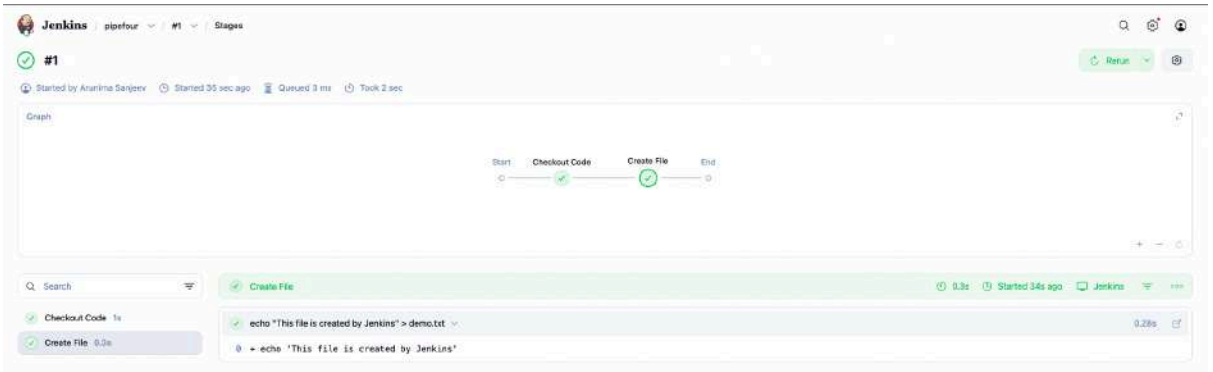
Download

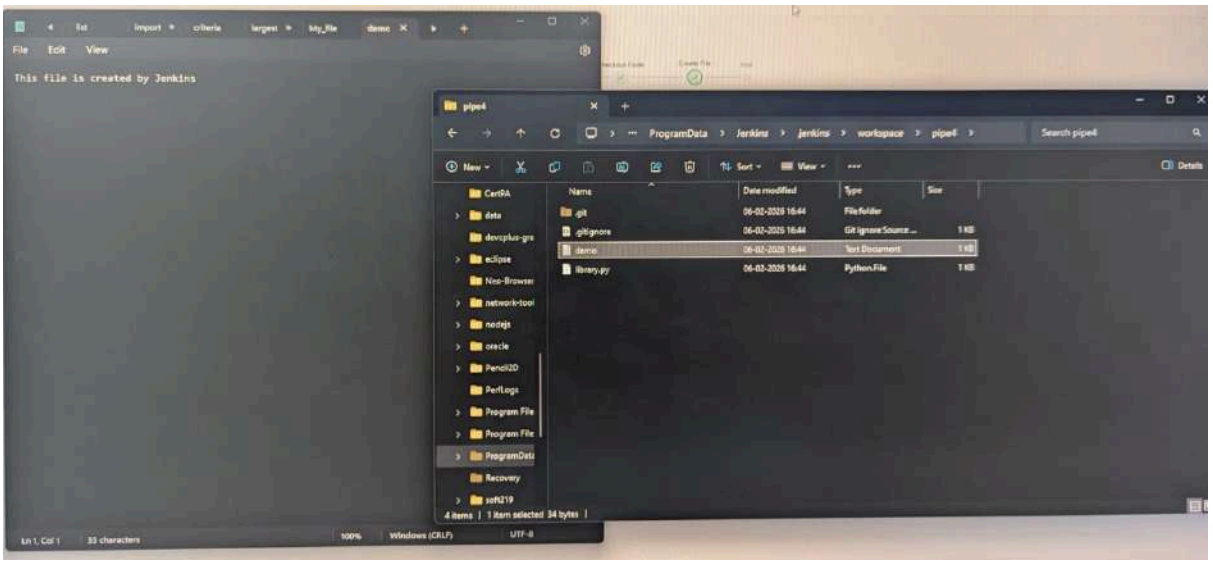
```
Started by user Arunima Sanjeev
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /Users/arunimasanjeev/.jenkins/workspace/pipethree
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Code)
[Pipeline] checkout
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/ArunimaSanjeev/StudentRegistrationModule
 > git init /Users/arunimasanjeev/.jenkins/workspace/pipethree # timeout=10
Fetching upstream changes from https://github.com/ArunimaSanjeev/StudentRegistrationModule
 > git --version # timeout=10
 > git --version # 'git version 2.39.2 (Apple Git-143)'
 > git fetch --tags --force --progress -- https://github.com/ArunimaSanjeev/StudentRegistrationModule +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/ArunimaSanjeev/StudentRegistrationModule # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5bb547553854d077a720dc361f5c5c5fd57370c9 (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 5bb547553854d077a720dc361f5c5c5fd57370c9 # timeout=10
Commit message: "Added .gitignore to ignore logs and temp files"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Print Message)
[Pipeline] echo
Hello! Jenkins Pipeline executed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Pipeline task 4:

Pipeline for Git Repository Checkout and Workspace File Creation







<u>Pipeline task 5:</u>

Pipeline for Git Repository Checkout and Reading File Contents

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script

try sample Pipeline...

```
pipeline {
    agent any
    stages {
        stage('Checkout Code') {
            steps {
                checkout scmGit(branches: [[name: '*/main']], extensions: [], userRemoteConfigs: [[url: 'https://github.com/ArunimaSanjeev/St
            }
        }

        stage('Read File') {
            steps {
                sh 'cat README.md'
            }
        }
    }
```

Use Groovy Sandbox

Jenkins / pipefive / #1

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#1'
- Timings
- Git Build Data
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces

#1 (07-Feb-2026, 12:25:22 pm)

Started by user Arunima Sanjeev

This run spent:

- 9 ms waiting;
- 11 sec build duration;
- 11 sec total from scheduled to completion.

git

Revision: 5bb547553854d077a720dc361f5c5c5fd57370c9
Repository: https://github.com/ArunimaSanjeev/StudentRegistrationModule

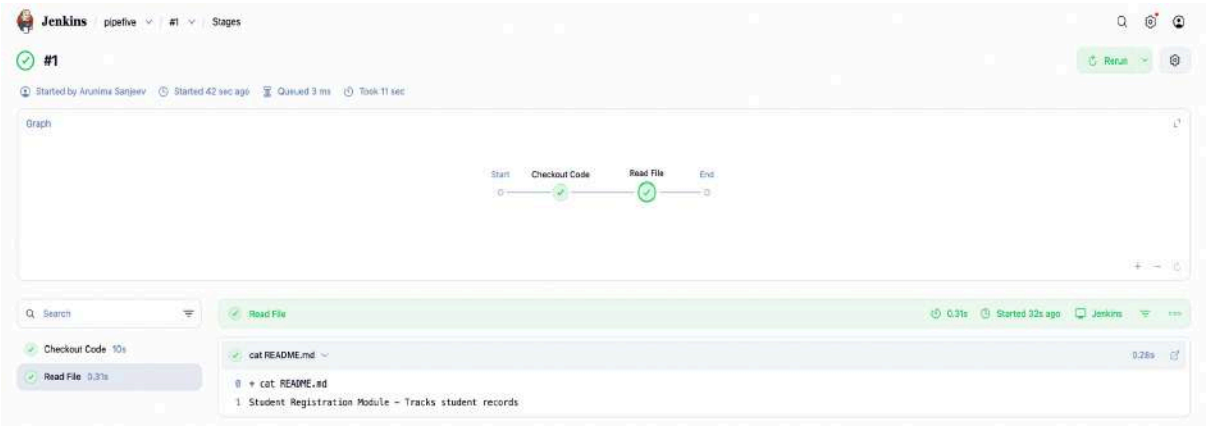
- refs/remotes/origin/main

No changes.

Console

Download Copy Vi

```
Started by user Arunima Sanjeev
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /Users/arunimasanjeev/.jenkins/workspace/pipefive
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Code)
[Pipeline] checkout
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/ArunimaSanjeev/StudentRegistrationModule
 > git init /Users/arunimasanjeev/.jenkins/workspace/pipefive # timeout=10
Fetching upstream changes from https://github.com/ArunimaSanjeev/StudentRegistrationModule
 > git --version # timeout=10
 > git --version # 'git version 2.39.2 (Apple Git-143)'
 > git fetch --tags --force --progress -- https://github.com/ArunimaSanjeev/StudentRegistrationModule +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/ArunimaSanjeev/StudentRegistrationModule # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5bb547553854d077a720dc361f5c5c5fd57370c9 (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 5bb547553854d077a720dc361f5c5c5fd57370c9 # timeout=10
Commit message: "Added .gitignore to ignore logs and temp files"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Read File)
[Pipeline] sh
+ cat README.md
Student Registration Module - Tracks student records
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```


Jenkins
pipefive
#1
Stages
#1
Rerun
Started by Arunima Sanjeev
Started 42 sec ago
Queued 3 ms
Took 11 sec
Graph
Start
Checkout Code
Read File
End
Search
Checkout Code 10s
Read File 0.31s
Read File
0.31s
Started 32s ago
Jenkins
cat README.md
0.28s
+ cat README.md
Student Registration Module - Tracks student records